

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum-590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

AJAY K O (1BM24CS025)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Ajay K O (**IBM24CS025**), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026 The Lab report has been approved as it satisfies the academic requirements in respect of aOPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. SeemaPatil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. (Any one) a) FCFS b) SJF c) Priority d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-16
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	17-20
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling.	21-27
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	28-31
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	32-35
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	36-39
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal.	40-42

Course Outcomes

C01	Applythe different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include<stdio.h> int main() {           int
i,j,n,avg_ct,avg_tat,avg_wt,time=0;      int
at[20],bt[20],ct[20],tat[20],wt[20],p[20];

    printf("enter the number of process:");
scanf("%d",&n);  for(i=0;i<n;i++){
    p[i]=i+1;    printf("enter
at%d :",i+1);
scanf("%d",&at[i]);
printf("enter bt%d :\n",i+1);
scanf("%d",&bt[i]);
}
    for(i=0;i<n-1;i++){
for(j=i+1;j<n;j++){
    if(at[i]>at[j]){
int temp;
temp=at[i];
at[i]=at[j];
at[j]=temp;
```

```

        temp=bt[i];
bt[i]=bt[j];        bt[j]=temp;

        temp=p[ i];
p[i]=p[j];        p[j]=temp;
    }
}
}
for(i=0;i<n;i++){
if(time<at[i]){
time=at[i];
}
ct[i]=time+bt[i];
tat[i]=ct[i]-at[i];
wt[i]=tat[i]-bt[i];
time=ct[i];
avg_ct+=ct[i];
avg_tat+=tat[i];
avg_wt+=wt[i];
}
avg_ct=avg_ct/n;
avg_tat=avg_tat/n;
avg_wt=avg_wt/n;

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");  for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],ct[i],tat[i],wt[i]);
}
printf("the avrages %d %d %d",avg_ct,avg_tat,avg_wt);
return 0;

```

OUTPUT :

```
enter the number of process:4
enter at1 :0
enter bt1 :
7
enter at2 :8
enter bt2 :
3
enter at3 :3
enter bt3 :
4
enter at4 :5
enter bt4 :
6

Process AT      BT      CT      TAT      WT
1      0      7      7      7      0
3      3      4      11     8      4
4      5      6      17     12     6
2      8      3      20     12     9
the avrages 13 9 4
```

=> SJ F(Non -pr ee mpt i ve) :

```
#include<stdio.h>
```

```
intmain() {
```

```
    int n,i,atat,awt,time=0,completed=0;
```

```
    int at[20],bt[20],ct[20],tat[20],wt[20],rt[20],p[20];
```

```
    intvisited[20]={0};
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
    p[i]=i+1;
```

```
        printf("Enter AT and BT for P%d: ",i+1);
```

```
    scanf("%d%d",&at[i],&bt[i]);
```

```
    }
```

```
    while(completed < n)
```

```
    {        int min_bt =
```

```
    9999;        int pos = -1;
```

```
    for(i=0;i<n;i++)
```

```
    {
```

```
        if(!visited[i] && at[i] <= time && bt[i] < min_bt)
```

```
        {
```

```
    min_bt = bt[i];
```

```
    pos = i;
```

```
        }
```

```
    }        if(pos
```

```
    == -1)
```



```

    {
time++;
continue;
    }

```

```

    rt[pos] = time - at[pos];
time+=bt[pos];      ct[pos] =
time;      tat[pos] = ct[pos] -
at[pos];      wt[pos]= tat[pos]
- bt[pos];      atat+=tat[pos];
awt+=wt[pos];

```

```

        visited[pos] = 1;
    completed++;
}
atat=atat/n;
awt=awt/n;

```

```
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
```

```
for(i=0;i<n;i++)
{
    print f( "%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\t",p[ i],a t[ i],bt[ i],c t[ i],ta t[ i],w t[ i],rt[ i]) ;
}

printf("the avrages %d %d",atat,awt);

return 0;
}
```

OUTPUT :

Enter number of processes: 5

Enter AT and BT for P1: 3 1

Enter AT and BT for P2: 1 4

Enter AT and BT for P3: 4 2

Enter AT and BT for P4: 0 6

Enter AT and BT for P5: 2 3

Process	AT	BT	CT	TAT	WT	RT
1	3	1	7	4	3	3
2	1	4	16	15	11	11
3	4	2	9	5	3	3
4	0	6	6	6	0	0
5	2	3	12	10	7	7

the avrages 8 4

=>SJF(pre-emptive)

```
#include <stdio.h>
```

```
int    main()    {                                int
n,atat,awt,i,time=0,completed=0,smallest;    int
at[20],bt[20],rt_bt[20];                                int
ct[20],tat[20],wt[20],rt[20];    int first_start[20];
```

```
    printf("Enter number of processes: ");
scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
    {
        printf("Enter AT and BT for P%d: ",i+1);
scanf("%d%d",&at[i],&bt[i]);
```

```
        rt_bt[i] = bt[i];
first_start[i] = -1;
    }
```

```
    while(completed < n)
    {        smallest = -
1;        int min =
9999;
for(i=0;i<n;i++)
    {
        if(at[i] <= time && rt_bt[i] > 0 && rt_bt[i] < min)
        {
            min = rt_bt[i];
smallest = i;
```

```

        }
    }

    if(smallest == -1)
    {
time++;
continue;
    }    if(first_start[smallest]
== -1)
    {        first_start[smallest] =
time;        rt[smallest] = time -
at[smallest];
    }

    rt_bt[smallest]--;
time++;

    if(rt_bt[smallest] == 0)
    {
        completed++;
ct[smallest] = time;

        tat[smallest] = ct[smallest] - at[smallest];
wt[smallest] = tat[smallest] - bt[smallest];
atat+=tat[smallest];        awt+=wt[smallest];
    }
}
atat=atat/n;
awt=awt/n;

```

```
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
```

```
for(i=0;i<n;i++)
```

```
{
```

```
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
```

```
    i+1,at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);
```

```
}
```

```
printf("the avrages %d %d",atat,awt);
```

```
return 0;
```

```
}
```

OUTPUT :

```
Enter number of processes: 5
```

```
Enter AT and BT for P1: 3 1
```

```
Enter AT and BT for P2: 1 4
```

```
Enter AT and BT for P3: 4 2
```

```
Enter AT and BT for P4: 0 6
```

```
Enter AT and BT for P5: 2 3
```

Process	AT	BT	CT	TAT	WT	RT
P1	3	1	4	1	0	0
P2	1	4	6	5	1	0
P3	4	2	8	4	2	2
P4	0	6	16	16	10	0
P5	2	3	11	9	6	6

```
the avrages 7 3
```

=> Priority (pre-emptive)

```
#include<stdio.h>
```

```
int main() {    float atat,awt;    int
n, i, time=0, completed=0, smallest;
at[20],bt[20],rt_bt[20];    int
pr[20],ct[20],tat[20],wt[20],rt[20]; int
first_start[20];
```

```
    printf("Enter number of processes: ");
scanf("%d",&n);
```

```
    for(i=0;i<n;i++)
    {
        printf("Enter AT and BT and PR for P%d: ",i+1);
scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
```

```
        rt_bt[i]= bt[i];
first_start[i]= -1;
    }
```

```
    while(completed < n)
    {        smallest = -
1;        int min =
9999;
for(i=0;i<n;i++)
    {
        if(at[i] <= time && rt_bt[i] > 0 && pr[i] < min)
        {            min
=            pr[i];
smallest = i;        }
    }
```

```

        if(smallest == -1)
        {
time++;
continue;
        }
        if(first_start[smallest] == -1)
        {
            first_start[smallest] = time;
rt[smallest] = time - at[smallest];
        }

        rt_bt[smallest]--;
time++;

        if(rt_bt[smallest] == 0)
        {
            completed++;
ct[smallest] = time;

            tat[smallest] = ct[smallest] - at[smallest];
wt[smallest] = tat[smallest] - bt[smallest];
atat+=tat[smallest];          awt+=wt[smallest];
        } }
atat=atat/n;
awt=awt/n;

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");  for(i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}
printf("the avrages TAT %f WT %f",atat,awt);
return0;

```

OUTPUT :

```
Enter number of processes: 5
Enter AT and BT and PR for P1: 0 3 5
Enter AT and BT and PR for P2: 2 2 3
Enter AT and BT and PR for P3: 3 5 2
Enter AT and BT and PR for P4: 4 4 4
Enter AT and BT and PR for P5: 6 1 1

Process AT    BT    PR    CT    TAT    WT    RT
P1      0      3      5    15    15    12    0
P2      2      2      3    10     8     6     0
P3      3      5      2     9     6     1     0
P4      4      4      4    14    10     6     6
P5      6      1      1     7     1     0     0
the avrages TAT 8.000000 WT 5.000000
```


=> Priority (Non-pre-emptive)

```
#include<stdio.h>
```

```
intmain() {
    float atat,awt;
    int n,i,time=0,completed=0;
    int at[20],bt[20],ct[20],tat[20],wt[20],rt[20],p[20],pr[20];
    intvisited[20]={0};

    printf("Enter number of processes: ");
    scanf("%d",&n);

    for(i=0;i<n;i++)
    {
        p[i]=i+1;
        printf("Enter AT and BT and PR for P%d: ",i+1);
        scanf("%d%d%d",&at[i],&bt[i],&pr[i]);
    }

    while(completed < n)
    {
        int min_pr =
9999;    int pos = -1;
        for(i=0;i<n;i++)
        {
            if(!visited[i] && at[i] <= time && pr[i] < min_pr)
            {
                min_pr = pr[i];
                pos = i;
            }
        }
        if(pos
== -1)
```

```

    {
time++;
continue;
    }

    rt[pos] = time - at[pos];
time+= bt[pos];

    ct[pos] = time;
    tat[pos] = ct[pos] - at[pos];
wt[pos] = tat[pos] - bt[pos];
atat+=tat[pos];
awt+=wt[pos];

    visited[pos] = 1;
completed++;
    }
atat=atat/n;
awt=awt/n;

printf("\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",p[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i],rt[i]);
}
printf("the avrages TAT %f WT %f",atat,awt);

return 0;
}

```

OUTPUT :

```
Enter number of processes: 5
Enter AT and BT and PR for P1: 0 3 5
Enter AT and BT and PR for P2: 2 2 3
Enter AT and BT and PR for P3: 3 5 2
Enter AT and BT and PR for P4: 4 4 4
Enter AT and BT and PR for P5: 6 1 1
```

Process	AT	BT	PR	CT	TAT	WT	RT
1	0	3	5	3	3	0	0
2	2	2	3	11	9	7	7
3	3	5	2	8	5	0	0
4	4	4	4	15	11	7	7
5	6	1	1	9	3	2	2

```
the avrages TAT 6.200000 WT 3.200000
```

=>Round Robin

```
#include <stdio.h>
```

```
int main() { float atat,awt; int n, i, time  
= 0, quantum; int at[20], bt[20],  
rem_bt[20]; int ct[20], tat[20], wt[20],  
rt[20], start[20]; int queue[100], front = 0,  
rear = 0;
```

```
printf("Enter number of processes: ");  
scanf("%d", &n);
```

```
for(i=0; i<n; i++) {  
    printf("Enter AT and BT for P%d: ", i+1);  
    scanf("%d%d", &at[i], &bt[i]);    rem_bt[i]  
=bt[i];    start[i] = -1;  
}
```

```
printf("Enter Time Quantum: ");  
scanf("%d", &quantum);
```

```
int remain = n; int  
visited[20] = {0};  
while(remain > 0) {  
    for(i=0; i<n; i++) {  
        if(at[i] <= time && rem_bt[i] > 0 && visited[i] == 0) {  
            queue[rear++] = i;    visited[i] = 1;  
        }  
    }  
}
```

```
if(front == rear) {  
    time++;  
    continue;
```

```

    }

    int idx = queue[front++];

    if(start[idx] == -1) {
start[idx] = time;
rt[idx] = time - at[idx];
    }

    if(rem_bt[idx] > quantum) {
time += quantum;
rem_bt[idx] -= quantum;
    }else{
        time+=
rem_bt[idx];        rem_bt[idx]
= 0;        ct[idx]=time;
tat[idx] = ct[idx] - at[idx];
wt[idx] = tat[idx] - bt[idx];
atat+=t at[ idx] ;
awt+=wt[idx];        remain--;
    }

    for(i=0; i<n; i++) {
        if(at[i] <= time && rem_bt[i] > 0 && visited[i] == 0) {
queue[rear++] = i;        visited[i] = 1;
        }
    }

    if(rem_bt[idx] > 0) {
        queue[rear++] = idx;
    } }

atat=atat/n;
awt=awt/n;

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for(i=0; i<n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",i+1, at[i], bt[i], ct[i], tat[i], wt[i], rt[i]);

```

```

    }
    printf("the avrages TAT %f WT %f",atat,awt);
return 0;
}

```

OUTPUT :

```

Enter number of processes: 5
Enter AT and BT for P1: 0 8
Enter AT and BT for P2: 5 2
Enter AT and BT for P3: 1 7
Enter AT and BT for P4: 6 3
Enter AT and BT for P5: 8 5
Enter Time Quantum: 3

Process AT      BT      CT      TAT      WT      RT
P1      0       8      22      22      14      0
P2      5       2      11       6       4       4
P3      1       7      23      22      15      2
P4      6       3      14       8       5       5
P5      8       5      25      17      12      9
the avrages TAT 15.000000 WT 10.000000

```

Program – 2 Question

:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE :

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void mlq_scheduling(int at[], int bt[], int q_no[], int n, int tq)
```

```
{ int rt[n], ct[n], tat[n], wt[n], completed[n]; int time = 0,
```

```
count = 0, last_q1 = -1; float avg_tat = 0, avg_wt = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        rt[i] = bt[i];
```

```
        completed[i] = 0;
```

```
    }
```

```
    while(count < n) {
```

```
        int idx = -1;
```

```
        RobinRound(int i = 1, Round++) { int
```

```
        j = (last_q1 + i) % n;
```

```
            if(q_no[j] == 1 && at[j] <= time && completed[j] == 0) {
```

```
                idx = j;          break;
```

```
            }
```

```
        }
```

```
        if(idx != -1) {
```

```

        int slice = (rt[idx] > tq) ? tq :
rt[idx];        time+=slice;        rt[idx]
-= slice;        last_q1 = idx;

        if(rt[idx] == 0) {
ct[idx] = time;
completed[idx] = 1;
count++;
        }
    }
    else {
        //Queue 2-FCFS
for(int i = 0; i < n; i++) {
        if(q_no[i]==2&&at[i]<=time && completed[i] == 0)
{
            idx=i;            break;
        }
    }

    if(idx != -1) {
rt[idx]--;        time++;
if(rt[idx] == 0) {
ct[idx] = time;
completed[idx] = 1;
count++;
        }
    } else {
time++;
    }
}
}

```



```

printf("\nPID\tQ_No\tAT\tBT\tCT\tTAT\tWT\n");
for(int i = 0; i < n; i++) {      tat[i] = ct[i] - at[i];
wt[i] = tat[i] - bt[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", i+1, q_no[i], at[i], bt[i], ct[i], tat[i],
wt[i]);      avg_tat += tat[i];      avg_wt += wt[i];
}
printf("\nAvg TAT: %.2f", avg_tat/n);
printf("\nAvg WT: %.2f\n", avg_wt/n);
}

```

```

intmain() {
    printf("Ahmad Lucky Gold\n");
    int n, tq = 2;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int at[n], bt[n], q_no[n];
    printf("Enter the Queue_No, AT and BT\n");
    for(int i = 0; i < n; i++) {      printf("p[%d] :
",i+1);
        scanf("%d %d %d", &q_no[i], &at[i], &bt[i]);
    }

    mlq_scheduling(at, bt, q_no, n, tq);

    return 0;
}

```

OUTPUT :

Enter the number of processes: 4

Enter the Queue_No, AT and BT

p[1] : 1 0 4

p[2] : 1 0 3

p[3] : 2 0 8

p[4] : 1 10 5

PID	Q_No	AT	BT	CT	TAT	WT
P1	1	0	4	6	6	2
P2	1	0	3	7	7	4
P3	2	0	8	20	20	12
P4	1	10	5	15	5	0

Avg TAT: 9.50

Avg WT: 4.50

Program – 3 Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: a)

Rate- Monotonic

b) Earliest-deadline First

CODE :

=> Rate- Monotonic

```
#include<stdio.h>
#include<math.h>
struct Process {    int
pid;    int burst;    int
period;            int
remaining_burst;
};
int gcd(int a, int b) {
if (b == 0) return a;
return gcd(b, a % b);
}
int lcm(int a, int b) {
return (a * b) / gcd(a, b);
} int main()
{    int n;
printf("Enter the number of processes: ");
scanf("%d", &n);

struct Process p[10];

printf("Enter the CPU burst times:\n");
for(int i = 0; i < n; i++) {    scanf("%d",
&p[i].burst);
p[i].pid = i + 1;
```

```

    }

    printf("Enter the time periods:\n");
    inthyperperiod = 1;    double
    utilization = 0;

    for(int i = 0; i < n; i++) {
scanf("%d",    &p[i].period);
p[i].remaining_burst = 0;
    hyperperiod = lcm(hyperperiod, p[i].period);
    utilization += (double)p[i].burst / p[i].period;
    }

    for(int i = 0; i < n - 1; i++){
for(int j = i + 1; j < n; j++){
if(p[i].period > p[j].period){
structProcess temp = p[i];
p[i]=p[j];        p[j]=temp;
        }
    }
}

    printf("LCM=%d\n", hyperperiod);
    printf("Rate Monotone Scheduling:\n");
    printf("PID\tBurst\tPeriod\n");    for(int i
=0;i<n; i++) {
        printf("%d\t%d\t%d\n", p[i].pid, p[i].burst, p[i].period);
    }

    double bound = n * (pow(2.0, 1.0 / n) - 1.0);
    printf("%f<=%f=>%s\n", utilization, bound,    (utilization <= bound)    ? "true" : "fa ls e");
    if(utilization<=1){    printf("Schedulable\n");
    }

```

```

printf("Scheduling occurs for %d ms\n", hyperperiod);

int current_process = -1;

for(int time = 0; time < hyperperiod; time++) {

    for(int i = 0; i < n; i++){
    if(time % p[i].period == 0){
    p[i].remaining_burst = p[i].burst;
        }
    }

    int next_process = -1;
    for(int i=0; i < n; i++){
    if(p[i].remaining_burst > 0) {
    next_process = i;          break;
        }
    }

    if(next_process != current_process) {
    if(next_process == -1) {
        printf("%dms onwards: CPU is idle\n", time);
    }else {
        printf("%dms onwards: Process %d running\n", time, p[next_process].pid);
    }
    current_process = next_process;
    }

    if(current_process != -1) {
        p[current_process].remaining_burst--;
    }
}

```

```
    return 0;
}
```

OUTPUT :

```
Enter the number of processes: 3
Enter the CPU burst times:
1 2 3
Enter the time periods:
4 6 12
LCM=12
Rate Monotone Scheduling:
PID    Burst  Period
1       1     4
2       2     6
3       3    12
0.833333<=0.779763=>false
Schedulable
Scheduling occurs for 12 ms
0ms onwards: Process 1 running
1ms onwards: Process 2 running
3ms onwards: Process 3 running
4ms onwards: Process 1 running
5ms onwards: Process 3 running
6ms onwards: Process 2 running
8ms onwards: Process 1 running
9ms onwards: Process 3 running
10ms onwards: CPU is idle
```

=> Earliest-deadline First

```
#include<stdio.h>
```

```
struct Process {  
    int pid;      int  
    burst;        int  
    deadline;     int  
    period;       int  
    remaining_bur  
    st;           int  
    absolute_deadl  
    ine;  
};
```

```
intgcd(int a, intb){  
    if(b== 0) returna;  
    returngcd(b, a %b);  
}
```

```
intlcm(int a, int b) {  
    return (a * b) / gcd(a,b);  
}
```

```
intmain() {  
    intn;  
    printf("Enter the number of processes: ");  
    scanf("%d", &n);
```

```
    struct Process p[10];  
    inthyperperiod = 1;  
    double utilization=0;
```

```

    for (int i = 0; i < n; i++) {
p[i].pid = i + 1;
        printf("Process %d - Enter Burst, Deadline, Period: ", p[i].pid);
scanf("%d %d %d", &p[i].burst, &p[i].deadline, &p[i].period);
p[i].remaining_burst = 0;

        hyperperiod = lcm(hyperperiod, p[i].period);
utilization += (double)p[i].burst / p[i].period;
    }

    printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", p[i].pid, p[i].burst, p[i].deadline, p[i].period);
    }
    if(utilization<=1){
printf("Schedulable\n");
    }
    printf("Scheduling occurs for %d ms\n", hyperperiod);

    for(int time = 0; time < hyperperiod; time++) {

        for (int i = 0; i < n; i++) {            if
(time    %    p[i].period    ==    0)    {
p[i].remaining_burst = p[i].burst;

                p[i].absolute_deadline = time + p[i].deadline;
            }
        }

        intmin_deadline = 999999;
intselected = -1;

        for(int i = 0; i < n; i++){
if(p[i].remaining_burst > 0){

```



```

        if (p[i].absolute_deadline < min_deadline) {
min_deadline= p[i].absolute_deadline;
selected = i;
        }
    }
}

if(selected == -1) {
    printf("%dms: CPU is idle.\n", time);
} else {
    printf("%dms: Task %d is running.\n", time, p[selected].pid);
p[selected].remaining_burst--;
    }
}

return 0;
}

```

OUTPUT :

```
Enter the number of processes: 3
Process 1 - Enter Burst, Deadline, Period: 3 7 20
Process 2 - Enter Burst, Deadline, Period: 2 4 5
Process 3 - Enter Burst, Deadline, Period: 2 8 10

PID    Burst  Deadline  Period
1       3      7         20
2       2      4         5
3       2      8        10
Schedulable
Scheduling occurs for 20 ms
0ms: Task 2 is running.
1ms: Task 2 is running.
2ms: Task 1 is running.
3ms: Task 1 is running.
4ms: Task 1 is running.
5ms: Task 3 is running.
6ms: Task 3 is running.
7ms: Task 2 is running.
8ms: Task 2 is running.
9ms: CPU is idle.
10ms: Task 2 is running.
11ms: Task 2 is running.
12ms: Task 3 is running.
13ms: Task 3 is running.
14ms: CPU is idle.
15ms: Task 2 is running.
16ms: Task 2 is running.
17ms: CPU is idle.
18ms: CPU is idle.
19ms: CPU is idle.
```

Program – 4

Question :

Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

Code :

=> producer-consumer

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#define N 5
```

```
int buf[N], in = 0, out = 0; sem_t
```

```
empty, full;
```

```
pthread_mutex_t m;
```

```
void* prod(void* p) {
```

```
int x = 0; while (1) {
```

```
sem_wait(&empty);
```

```
pthread_mutex_lock(&m);
```

```
buf[in] = x; printf("Produced: %d at
```

```
buffer[%d]\n", x++, in);
```

```
in = (in + 1) % N;
```

```
pthread_mutex_unlock(&m);
```

```
sem_post(&full);
```

```
}
```

```
}
```

```
void* cons(void* p) {
```

```
int x; while (1) {
```

```
sem_wait(&full);
```

```
pthread_mutex_lock(&m);
```

```
x = buf[out]; printf("Consumed: %d from
```

```
buffer[%d]\n", x, out);
```

```
out = (out + 1) % N;
```

```

        pthread_mutex_unlock(&m);
        sem_post(&empty);
    }
}

intmain() {
    pthread_t p, c;

    sem_init(&empty, 0, N);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&m, NULL);

    pthread_create(&p, NULL, prod, NULL);
    pthread_create(&c, NULL, cons, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);
}

```

OUTPUT :

```

Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]

```

=> Dining Philosophers

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
pthread_mutex_t forks[N];
```

```
void* phil(void* n) { int id
```

```
=(int*)n; int L = id, R =
```

```
(id+ 1) % N;
```

```
while (1){ printf("P%d  
thinking\n", id); sleep(1);
```

```
if (id % 2 == 0) {  
pthread_mutex_lock(&forks[L]);  
pthread_mutex_lock(&forks[R]);  
} else {  
pthread_mutex_lock(&forks[R]);  
pthread_mutex_lock(&forks[L]);  
}
```

```
printf("P%d eating\n", id);  
sleep(2);
```

```
pthread_mutex_unlock(&forks[L]);  
pthread_mutex_unlock(&forks[R]);  
}  
}
```

```

int main() {
pthread_t t[N];
intid[N];

    for(int i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
id[i]=i;
        pthread_create(&t[i], NULL, phil, &id[i]);
    }

    for(int i = 0; i < N; i++)
pthread_join(t[i], NULL);
}

```

OUTPUT :

```

P0 thinking
P1 thinking
P2 thinking
P3 thinking
P4 thinking
P3 eating
P0 eating
P0 thinking
P2 eating
P3 thinking
P4 eating
P2 thinking
P1 eating
P4 thinking
P2 eating

```

Program – 5

Question:

Write a C program to simulate: (Any one)

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

CODE :

=> Banker's

```
#include<stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k, alloc[10][10], max[10][10], avail[10], f[10]={0}, ans[10], ind=0, flag;
```

```
    printf("Enter number of processes: "); scanf("%d", &n);
```

```
    printf("Enter number of resources: "); scanf("%d", &m);
```

```
    printf("Enter Allocation Matrix:\n");
```

```
    for(i=0; i<n; i++) for(j=0; j<m; j++) scanf("%d", &alloc[i][j]);
```

```
    printf("Enter Maximum Demand Matrix:\n");
```

```
    for(i=0; i<n; i++) for(j=0; j<m; j++) scanf("%d", &max[i][j]);
```

```
    printf("Enter Available Resources:\n");
```

```
    for(j=0; j<m; j++) scanf("%d", &avail[j]);
```

```
    // Banker's Algorithm Logic
```

```
    for(k=0; k<n; k++) {
```

```
        for(i=0; i<n; i++) {
```

```
            if(!f[i]) {          flag=0;
```

```
            for(j=0; j<m; j++) {
```

```
                if(max[i][j] - alloc[i][j] > avail[j]) flag=1;
```

```
            }
```

```

        if(!flag) {
ans[ind++]=i;
        for(j=0; j<m; j++) avail[j] += alloc[i][j];
f[i]=1;
        }
    }
}
}

//NEW:Check if all processes made it into the safe sequence
if(ind == n) {
    printf("System is in a safe state.\nSafe sequence is: ");
for(i=0;i<n;i++) {
        printf("P%d%s", ans[i], " -> ");
    }
} else {
    printf("System is in an UNSAFE state. Deadlock may occur.\n");
}

return 0;
}

```

OUTPUT :

```

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2
System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2 ->

```


=>DeadlockDetection

```
#include <stdio.h>
```

```
intmain(){    intn,m,i,j,k,alloc[10][10],req[10][10],avail[10], f[10]={0}, ans[10],  
ind=0, flag;
```

```
    printf("Enterthenumberofprocesses:");scanf("%d",&n);  
    printf("Enterthenumberofresources:");scanf("%d",&m);
```

```
    printf("Entertheallocationmatrix:\n");    for(i=0;i<n;i++)  
for(j=0; j<m; j++) scanf("%d", &alloc[i][j]);
```

```
    printf("Entertherequestmatrix:\n");    for(i=0;i<n;i++)  
for(j=0; j<m; j++) scanf("%d", &req[i][j]);
```

```
    printf("Enter the available resources:\n");  
    for(j=0; j<m; j++) scanf("%d", &avail[j]);
```

```
    // Deadlock Detection Logic  
for(k=0; k<n; k++) {  
    for(i=0; i<n; i++) {  
        if(!f[i]) {  
            flag=0;  
            //Checkifprocessrequestcanbemetybyavailable resources  
for(j=0; j<m; j++) {  
                if(req[i][j] > avail[j]) flag=1;  
            }  
            //Ifyes,processexecutesandreleaseitsallocated resources  
if(!flag){                ans[ind++]=i;                for(j=0;j<m; j++)  
avail[j]+= alloc[i][j];                f[i]=1;  
            }  
        }  
    }  
}
```

```
    // Output formatting if(ind == n) {  
printf("System is in safe state.\nSafe Sequence is: ");  
for(i=0; i<n; i++) {  
    printf("P%d%s", ans[i], " ");  
}  
} else {  
    printf("Deadlock Detected in the system!\n");  
printf("The deadlocked processes are: ");
```

```

        //Loopthrough the finish array to findthe0s
for(i=0;i<n;i++) {          if(f[i] == 0) {
printf("P%d",i);
        }
}
    printf("\n");
    }return0;
}

```

OUTPUT :

```

Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter the request matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2
Enter the available resources:
0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

```

Program – 6

Question :

Write a C program to simulate the following contiguous memory allocation techniques. (Anyone)

a) Worst-fit

b) Best-fit

c) First-fit

CODE :

=> Worst-fit, Best-fit and First-fit

```
#include <stdio.h>
```

```
void FF(int p[], int n, int b[], int m) {  
    int used[m];
```

```
    for(int i=0; i<m; i++)  
        used[i] = 0;
```

```
    printf("\n--- First Fit Allocation ---\n");
```

```
    for(int i=0; i<n; i++) {  
        int alloc = -1;
```

```
        for(int j=0; j<m; j++) {  
            if(!used[j] && b[j] >= p[i]) {  
                alloc = j;  
                used[j] = 1;  
                break;  
            }  
        }
```

```
        if(alloc != -1)        printf("Process %d  
(%d) -> Block %d\n",          i+1, p[i],  
alloc+1);        else  
            printf("Process %d (%d) -> Not Allocated\n",  
                i+1, p[i]);  
    }
```

```
void BF(int p[], int n, int b[], int m) {  
    int used[m];
```

```

for(int i=0; i<m; i++)
used[i] = 0;    printf("\n---
Best Fit Allocation ---\n");

for(int i=0; i<n; i++) {
    int idx = -1;

    for(int j=0; j<m; j++) {
if(!used[j] && b[j] >= p[i] &&
(idx == -1 || b[j] < b[idx])) {
        idx = j;
    }
}

if(idx != -1) {
    used[idx] = 1;

    printf("Process %d (%d) -> Block %d\n",
i+1, p[i], idx+1);
}
else {    printf("Process %d (%d) -> Not
Allocated\n",
        i+1, p[i]);
}
}
}

void WF(int p[], int n, int b[], int m) {
int used[m];

for(int i=0; i<m; i++)
    used[i] = 0;

printf("\n--- Worst Fit Allocation ---\n");

for(int i=0; i<n; i++) {
    int idx = -1;

    for(int j=0; j<m; j++) {
if(!used[j] && b[j] >= p[i] &&
(idx == -1 || b[j] > b[idx])) {
        idx = j;
    }
}
}
}

```

```

        if(idx != -1) {
            used[idx] = 1;

            printf("Process%d(%d)-> Block %d\n",
i+1, p[i], idx+1);
        }
        else{
            printf("Process %d (%d) -> Not
Allocated\n",
            i+1, p[i]);
        }
    }
}

int main() {
    int m, n;

    printf("Enternumberofmemory blocks: ");
    scanf("%d", &m);

    intblocks[m],b1[m],b2[m],b3[m];

    printf("Entersizeofeachmemory block:\n");

    for(int i=0; i<m; i++) {
        printf("Block %d: ", i+1);
        scanf("%d", &blocks[i]);

        b1[i]=b2[i]=b3[i]=blocks[i];
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    int p[n];

    printf("Entersizeofeachprocess:\n");

    for(int i=0; i<n; i++) {
        printf("Process %d: ", i+1);
        scanf("%d", &p[i]);
    }

    FF(p, n, b1, m);

```

```
    BF(p, n, b2, m);  
    WF(p, n, b3, m);  
  
    return 0;  
}
```

OUTPUT :

```
Enter number of memory blocks: 5  
Enter size of each memory block:  
Block 1: 100  
Block 2: 500  
Block 3: 200  
Block 4: 300  
Block 5: 600  
  
Enter number of processes: 4  
Enter size of each process:  
Process 1: 212  
Process 2: 417  
Process 3: 112  
Process 4: 426  
  
--- First Fit Allocation ---  
Process 1 (212) -> Block 2  
Process 2 (417) -> Block 5  
Process 3 (112) -> Block 3  
Process 4 (426) -> Not Allocated  
  
--- Best Fit Allocation ---  
Process 1 (212) -> Block 4  
Process 2 (417) -> Block 2  
Process 3 (112) -> Block 3  
Process 4 (426) -> Block 5  
  
--- Worst Fit Allocation ---  
Process 1 (212) -> Block 5  
Process 2 (417) -> Block 2  
Process 3 (112) -> Block 4  
Process 4 (426) -> Not Allocated
```

Program -7

Question:

Write a C program to simulate page replacement algorithms. a)

FIFO

b) LRU

c) Optimal

CODE:

=> FIFO, LRU and Optimal

```
#include <stdio.h>
```

```
int n, m, p[99];
```

```
void disp(int f[]) {    for (int i = 0; i < n; i++) f[i] == -1 ? printf("- ") :  
printf("%d ", f[i]); printf("\n");  
}
```

```
void fifo() {    int f[20], pf = 0, nxt  
= 0;    for (int i = 0; i < n; i++) f[i]  
= -1; printf("\n--- FIFO ---\n");  
for (int i = 0; i < m; i++) {  
    int hit = 0;    for (int j = 0; j < n; j++) if (f[j] ==  
p[i]) hit = 1;    if (!hit) { f[nxt] = p[i]; nxt = (nxt + 1)  
% n; pf++; }    printf("Page %d -> ", p[i]); disp(f);  
}  
printf("Total Page Faults: %d\n", pf);  
}
```

```
void opt() {    int f[20], pf = 0;  
for (int i = 0; i < n; i++) f[i] = -1;  
printf("\n--- OPTIMAL ---\n");  
for (int i = 0; i < m; i++) {  
    int hit = 0;    for (int j = 0; j < n; j++) if (f[j]  
== p[i]) hit = 1;    if (!hit) {    int rep = 0,  
far = -1;    for (int j = 0; j < n; j++) {  
if (f[j] == -1) { rep = j; break; }  
    int k;    for (k = i + 1; k < m; k++) if  
(p[k] == f[j]) break;    if (k > far) { far = k; rep = j;  
}  
    }  
    f[rep] = p[i]; pf++;  
}
```

```

    }
    printf("Page %d -> ", p[i]); disp(f);
}
printf("Total Page Faults: %d\n", pf);
}

void lru() { int f[20], t[20] = {0}, pf = 0; for
(int i = 0; i < n; i++) f[i] = -1; printf("\n--- LRU
---\n"); for (int i = 0; i < m; i++) { int hit = -
1; for (int j = 0; j < n; j++) if (f[j] == p[i]) hit
= j; if (hit != -1) t[hit] = i; else {
int rep = 0, min = 999; for (int j = 0; j < n;
j++) { if (f[j] == -1) { rep = j; break; }
if (t[j] < min) { min = t[j]; rep = j; }
}
f[rep] = p[i]; t[rep] = i; pf++;
}
printf("Page %d -> ", p[i]); disp(f);
}
printf("Total Page Faults: %d\n", pf);
}

int main() { printf("Enter number of frames: ");
scanf("%d", &n); printf("Enter number of pages: ");
scanf("%d", &m); printf("Enter reference string: ");
for (int i = 0; i < m; i++) scanf("%d", &p[i]);
fifo();
opt();
lru();
return 0;
}

```

OUTPUT :


```

Enter number of frames: 3
Enter number of pages: 21
Enter reference string: 5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0

```

```

--- FIFO ---

```

```

Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 3 1
Page 0 -> 2 3 0
Page 2 -> 2 3 0
Page 4 -> 4 3 0
Page 3 -> 4 3 0
Page 5 -> 4 3 0
Page 2 -> 4 2 0
Page 0 -> 4 2 0
Page 2 -> 4 2 0
Page 1 -> 4 2 1
Page 2 -> 4 2 1
Page 7 -> 7 2 1
Page 0 -> 7 0 1
Page 1 -> 7 0 1
Page 1 -> 7 0 1
Page 0 -> 7 0 1
Total Page Faults: 11

```

```

--- OPTIMAL ---

```

```

Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 4 -> 2 4 3
Page 2 -> 2 4 3
Page 3 -> 2 4 3
Page 2 -> 2 4 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 1 -> 2 0 1
Page 2 -> 2 0 1
Page 7 -> 7 0 1
Page 0 -> 7 0 1
Page 1 -> 7 0 1
Page 1 -> 7 0 1
Page 0 -> 7 0 1
Total Page Faults: 9

```

```

--- LRU ---

```

```

Page 5 -> 5 - -
Page 0 -> 5 0 -
Page 1 -> 5 0 1
Page 0 -> 5 0 1
Page 2 -> 2 0 1
Page 3 -> 2 0 3
Page 0 -> 2 0 3
Page 2 -> 2 0 3
Page 4 -> 2 0 4
Page 3 -> 2 3 4
Page 2 -> 2 3 4
Page 2 -> 2 3 4
Page 0 -> 2 3 0
Page 2 -> 2 3 0
Page 1 -> 2 1 0
Page 2 -> 2 1 0
Page 7 -> 2 1 7
Page 0 -> 2 0 7
Page 1 -> 1 0 7
Page 1 -> 1 0 7
Page 0 -> 1 0 7
Total Page Faults: 12

```